

# Equipo BCEL / SGA Escuela – Entrega 3

## Prueba de compilación: Secciones 6.1–6.4

Aplicaciones Distribuidas (ISR-701)  
UTEQ

29 de julio de 2026

### Índice

|                                                                                    |           |
|------------------------------------------------------------------------------------|-----------|
| <b>1. Resumen</b>                                                                  | <b>3</b>  |
| <b>2. Introducción</b>                                                             | <b>5</b>  |
| <b>3. Trabajos relacionados</b>                                                    | <b>6</b>  |
| 3.1. Sistemas de bases de datos distribuidas y su evolución hacia NewSQL . . . . . | 6         |
| 3.2. CockroachDB y el consenso distribuido . . . . .                               | 7         |
| 3.3. Procesamiento paralelo con Apache Spark . . . . .                             | 7         |
| 3.4. Leyes de escalabilidad: Amdahl y Gustafson . . . . .                          | 7         |
| <b>4. Fundamento teórico</b>                                                       | <b>8</b>  |
| 4.1. Modelo de bases de datos distribuidas . . . . .                               | 8         |
| 4.2. Teorema CAP y modelo PACELC . . . . .                                         | 9         |
| 4.3. Consenso distribuido: Raft . . . . .                                          | 10        |
| 4.4. Modelo de programación paralela y leyes de escalabilidad . . .                | 10        |
| <b>5. Diseño del esquema distribuido</b>                                           | <b>11</b> |
| 5.1. Estrategia de fragmentación efectivamente adoptada . . . . .                  | 11        |
| 5.2. Registro de decisión arquitectónica (ADR-003) . . . . .                       | 12        |
| 5.3. Seguridad de credenciales . . . . .                                           | 13        |
| <b>6. Verificación experimental de la arquitectura implementada</b>                | <b>13</b> |
| 6.1. Contrato gRPC compartido . . . . .                                            | 13        |
| 6.2. Autenticación JWT de single sign-on . . . . .                                 | 14        |

|                                                                               |           |
|-------------------------------------------------------------------------------|-----------|
| 6.3. Prueba de independencia del canal gRPC respecto de la API REST . . . . . | 15        |
| <b>7. Pipeline analítico paralelo</b>                                         | <b>16</b> |
| <b>8. Protocolo y resultados</b>                                              | <b>16</b> |
| 8.1. Alcance de la medición realizada . . . . .                               | 16        |
| 8.2. Resultados observados . . . . .                                          | 17        |
| 8.3. Protocolo formal pendiente . . . . .                                     | 17        |
| <b>9. Discusión</b>                                                           | <b>17</b> |
| 9.1. Limitaciones de esta entrega . . . . .                                   | 18        |
| 9.2. Amenazas a la validez . . . . .                                          | 18        |
| 9.3. Relación con la Entrega 2 . . . . .                                      | 19        |
| <b>10. Conclusiones e integración con Entrega 4</b>                           | <b>19</b> |

# 1. Resumen

Este trabajo documenta el estado de la capa de datos distribuida y la comunicación entre microservicios del Sistema de Gestión Académica (SGA), compuesto por cuatro microservicios (**sga-principal**, **micro-docente**, **sga-secretaria**, **soporte-técnico**) que persisten en una instancia única de PostgreSQL en AWS EC2, fragmentada verticalmente en esquemas por servicio. Como **método**, se consolidó la comunicación entre **sga-principal** y **micro-docente** mediante un contrato gRPC compartido con autenticación JWT de *single sign-on* en los cuatro portales, verificando experimentalmente que el canal gRPC (9091/9092) opera de forma independiente de la API REST de docente. Como **resultado**, se demuestra el flujo completo

React  $\rightarrow$  sga-principal  $\rightarrow$  gRPC  $\rightarrow$  micro-docente  $\rightarrow$  BD

para el registro de asistencia de un curso completo, con latencia media de 51 ms frente a 145 ms de la ruta REST equivalente. Se identifica como **limitación** explícita la ausencia de un clúster distribuido con fragmentación horizontal, replicación y consenso (CockroachDB/Raft), así como del pipeline analítico en Apache Spark exigidos por la guía; ambos se planifican como prioridad de la Entrega 4. Se **concluye** que la arquitectura gRPC+JWT alcanzada constituye una base de comunicación distribuida sólida, sobre la cual evaluar posteriormente el impacto de la fragmentación real de datos.

**Palabras clave:** sistemas de bases de datos distribuidas; fragmentación horizontal; CockroachDB; consenso Raft; Apache Spark; Ley de Amdahl.

**Abstract.** This work documents the state of the distributed data layer and inter-service communication of the Academic Management System (SGA), composed of four microservices (**sga-principal**, **micro-docente**, **sga-secretaria**, **soporte-técnico**) that persist on a single AWS-hosted PostgreSQL instance, vertically fragmented into per-service schemas. As **method**, the team consolidated a shared gRPC contract between **sga-principal** and **micro-docente**, secured with single-sign-on JWT authentication across all four portals, and experimentally verified that the gRPC channel (9091/9092) operates independently of the teacher microservice's REST API. As **result**, the complete flow

React  $\rightarrow$  gRPC  $\rightarrow$  micro-docente  $\rightarrow$  database

for group attendance registration is demonstrated, with a mean latency of 51 ms versus 145 ms for the equivalent REST path. This delivery explicitly did not reach a horizontally fragmented, replicated, consensus-based cluster

(e.g. CockroachDB with Raft) nor the required Apache Spark analytical pipeline, both required by the course guide; these are documented as open **limitations** and prioritized for the next delivery. We **conclude** that the gRPC+JWT communication layer achieved is a solid, verified foundation on which horizontal fragmentation and consensus impact can later be evaluated.

## 2. Introducción

El Sistema de Gestión Académica (SGA) construido por el equipo a lo largo del PFC digitaliza los procesos de una unidad educativa: matriculación de estudiantes, registro de calificaciones por trimestre, trámites de secretaría y soporte técnico interno. En la Entrega 1 (E1) se estableció el dominio y los actores del sistema (estudiante, representante, docente, secretaría, soporte técnico) y se prototipó la comunicación entre procesos mediante gRPC bajo el modelo lógico de Lamport, decisión que se consolidó como el mecanismo de comunicación interna definitivo del sistema: `sga-principal` expone un servicio gRPC (`PrincipalGrpcService`) consumido por Secretaría y Soporte, y a su vez actúa como cliente gRPC del microservicio `micro-docente` para todo lo relativo a calificaciones, según el siguiente flujo:

`React → HTTP → sga-principal → gRPC → micro-docente → BD`

En la Entrega 2 (E2) esa arquitectura se materializó como cuatro microservicios independientes y heterogéneos en su stack tecnológico: `sga-principal` (Java 21, Spring Boot 3.2, autenticación JWT según RFC 7519 [1]), `micro-docente` (Python/Django, servidor gRPC de calificaciones), `sga-secretaria` (Java 21, Spring Boot 3.2) y `soporte-técnico` (gestión de incidencias), contenerizados con Docker Compose siguiendo el estilo arquitectónico de microservicios descrito por Fielding [2] y Newman [3]. La persistencia de E2, sin embargo, quedó resuelta de forma provisional: una única instancia PostgreSQL 17 aloja simultáneamente los esquemas lógicos `sga_principal` y `sga_secretaria` en un mismo contenedor sin fragmentación, sin réplica y sin tolerancia a fallos — un punto único de falla (*SPOF*) que, además, obliga a que `sga-principal` sea el único escritor autorizado del esquema `sga_secretaria` para preservar la integridad referencial entre ambos dominios.

**Problema abordado en E3.** La guía de la asignatura exige sustituir la persistencia mono-nodo de E2 por un clúster distribuido con fragmentación horizontal, replicación y consenso (CockroachDB/Raft), y complementarla con un pipeline analítico paralelo en Apache Spark. Por restricciones de tiempo del equipo, esta entrega no alcanzó a completar esos dos componentes: la persistencia continúa sobre una instancia única de PostgreSQL en AWS EC2, fragmentada únicamente de forma vertical (un esquema por microservicio), sin réplicas y sin tolerancia a la caída del nodo de base de datos. Lo que sí se consolidó y se verificó experimentalmente en esta entrega es la capa de comunicación distribuida entre servicios: el contrato gRPC compartido entre `sga-principal` y `micro-docente`, la autenticación JWT de *single sign-on* entre los cuatro portales, y la independencia del canal gRPC respecto de

la disponibilidad de la API REST de docente (Sección 6). Estas piezas — fragmentación horizontal real, replicación con Raft y pipeline PySpark — se documentan como limitaciones abiertas en la Sección 9 y se planifican como el trabajo prioritario de la Entrega 4.

Las **contribuciones** concretas de este trabajo son: (1) la consolidación de un contrato **gRPC** compartido entre **sga-principal** y **micro-docente** para asistencias y calificaciones, con autenticación **JWT** de *single sign-on* verificada en los cuatro portales del sistema; (2) la verificación experimental — con evidencia audiovisual y de bitácora — de que el canal **gRPC** opera de forma independiente de la disponibilidad de la API REST del microservicio docente; (3) la medición comparativa de latencia entre las rutas **gRPC** y **REST** para las operaciones más frecuentes del dominio (registro de asistencia grupal, consulta de asignaciones); y (4) la identificación explícita, con su correspondiente plan de mitigación, de las carencias de esta entrega en fragmentación horizontal, replicación/consenso y procesamiento paralelo, que se documentan en la Sección 9 como la hoja de ruta prioritaria hacia la Entrega 4.

## 3. Trabajos relacionados

### 3.1. Sistemas de bases de datos distribuidas y su evolución hacia NewSQL

Özsu y Valduriez [4] sistematizan los principios de fragmentación, replicación y transparencia de ubicación que fundamentan cualquier sistema de base de datos distribuido (SBDD), estableciendo las reglas de corrección — completitud, reconstrucción y disjunción — que debe cumplir toda estrategia de fragmentación horizontal. Kleppmann [5] extiende esa discusión hacia los sistemas de datos modernos, enfatizando el compromiso entre correctitud y rendimiento cuando la partición de datos se combina con réplicas asíncronas o basadas en consenso. Ambas obras anteceden a la familia de sistemas *NewSQL*, que busca conservar las garantías transaccionales de las bases relacionales clásicas a la escala horizontal de los sistemas *NoSQL*: Corbett et al. [6] presentaron con Spanner el primer sistema NewSQL geo-distribuido con consistencia externa verificable en producción, mientras que arquitecturas previas orientadas a disponibilidad, como Dynamo [7], ilustran el extremo opuesto del espacio de diseño — disponibilidad y partición sobre consistencia fuerte — frente al cual se contrasta la decisión de diseño adoptada en este trabajo.

### 3.2. CockroachDB y el consenso distribuido

CockroachDB [8] es la base de datos distribuida adoptada como referencia teórica en esta entrega, aunque — como se explica en la Sección 9 — no llegó a desplegarse en producción dentro del plazo de E3. Sus autores documentan un diseño que fragmenta automáticamente cada tabla en *rangos* de aproximadamente 512 MiB y replica cada rango mediante un grupo Raft independiente, ofreciendo consistencia serializable de extremo a extremo sin coordinador centralizado. El algoritmo de consenso subyacente, Raft [9], fue diseñado explícitamente para ser más comprensible que Paxos sin sacrificar sus garantías de seguridad, descomponiendo el problema en elección de líder, replicación de registros (*log*) y una regla de compromiso por mayoría. El comportamiento de CockroachDB ante particiones de red se explica mediante el teorema CAP formulado por Brewer [10] y demostrado formalmente por Gilbert y Lynch [11], y se refina con el modelo PACELC de Abadi [12], que añade el eje de *latencia* incluso en ausencia de particiones: bajo PACELC, CockroachDB se clasifica como PC/EC, priorizando consistencia tanto ante particiones como en operación normal, a costa de la latencia adicional que introduce el consenso por mayoría en cada escritura.

### 3.3. Procesamiento paralelo con Apache Spark

El modelo de *Resilient Distributed Dataset* (RDD) introducido por Zaharia et al. [13] resolvió el principal cuello de botella de MapReduce para cargas de trabajo iterativas — la materialización de resultados intermedios en disco — mediante una abstracción de colección distribuida inmutable con linaje de transformaciones y evaluación perezosa. Chambers y Zaharia [14] documentan la evolución de esa abstracción hacia **DataFrames** con optimización de consultas (*Catalyst*), que sería la interfaz a usar en el pipeline analítico del equipo una vez implementado (`pyspark.sql`). Salloum et al. [15] realizan una revisión sistemática de aplicaciones de *big data analytics* sobre Spark, reportando que la mayoría de los casos de estudio en dominios transaccionales — como el académico de este PFC — combinan filtrado, agregación por ventanas y algún componente de aprendizaje automático ligero, exactamente el patrón de transformaciones exigido por el Módulo E de esta guía y que el equipo planifica para la Entrega 4.

### 3.4. Leyes de escalabilidad: Amdahl y Gustafson

La Ley de Amdahl [16] acota el *speedup* máximo alcanzable al asumir un tamaño de problema fijo, mientras que Gustafson [17] argumenta que,

en la práctica, el volumen de datos procesado escala junto con los recursos disponibles, por lo que la fracción serial efectiva disminuye con  $N$  en lugar de mantenerse constante. Jain [18] y Wohlin et al. [19] proveen el marco metodológico — diseño experimental, repeticiones, intervalos de confianza y amenazas a la validez — que este trabajo intenta seguir para el contraste, informal por ahora, entre latencia gRPC y REST reportado en la Sección 8, y que deberá aplicarse con rigor completo al pipeline analítico en la Entrega 4.

## 4. Fundamento teórico

### 4.1. Modelo de bases de datos distribuidas

Un sistema de base de datos distribuido (SBDD) es un conjunto de múltiples bases de datos lógicamente relacionadas, gestionadas por un único sistema y percibidas por el usuario como una única base [4]. Sus objetivos de diseño son la transparencia de fragmentación, de replicación y de ubicación; la autonomía local de cada nodo; y la escalabilidad horizontal. La *fragmentación horizontal* de una relación  $R$  produce fragmentos  $R_1, R_2, \dots, R_n$  mediante selección sobre un atributo particionador:

$$R_i = \sigma_{p_i}(R), \quad i = 1, \dots, n, \quad (1)$$

donde  $\{p_i\}$  es un conjunto de predicados que debe satisfacer tres reglas de corrección [4]: *completitud* (todo registro de  $R$  pertenece a algún  $R_i$ ), *reconstrucción* ( $R = \bigcup_i R_i$ ) y *disjunción* ( $R_i \cap R_j = \emptyset$  para  $i \neq j$ ). En el caso de estudio de este trabajo,  $R$  es la relación `calificaciones` del microservicio `micro-docente` y el atributo particionador planificado es `id_periodo` (el trimestre dentro de un año lectivo, gestionado por la entidad `PeriodoEvaluacion de sga-principal`), de modo que cada predicado  $p_i$  selecciona las calificaciones de un único período de evaluación — la unidad temporal sobre la que operan casi todas las consultas del sistema real (promedio formativo y final por trimestre, reportes de rendimiento por período). La *colocalización* adicional de `calificaciones` respecto de `matrículas` por `id_estudiante` (o, de forma equivalente, `id_matricula`) evitaría *joins* distribuidos entre nodos al resolver el patrón de acceso más frecuente del dominio: obtener las calificaciones de un estudiante concreto en un trimestre concreto. La implementación de esta fragmentación sobre CockroachDB se realizaría mediante cláusulas `PARTITION BY RANGE`, según documenta el manual oficial del motor [20]. Como se detalla en la Sección 5, en E3 el equipo aplicó únicamente *fragmentación vertical* (un esquema por microservicio) y no llegó a implementar esta fragmentación horizontal por `id_periodo`.



La *replicación* multiplica cada fragmento en  $f$  copias para tolerar la caída de nodos. Un grupo de réplicas de tamaño  $f$  tolera la pérdida simultánea de hasta

$$t_{\text{máx}} = \left\lfloor \frac{f-1}{2} \right\rfloor \quad (2)$$

réplicas, siempre que se mantenga un quórum de escritura mayoritario de

$$q = \left\lfloor \frac{f}{2} \right\rfloor + 1 \quad (3)$$

réplicas activas [8,9]. Con el factor de replicación por defecto  $f = 3$  que la guía exige adoptar en el clúster CockroachDB de la Entrega 4,  $t_{\text{máx}} = 1$  y  $q = 2$ : el sistema tolerará la caída de un nodo sin perder disponibilidad de escritura, pero perderá disponibilidad si caen dos de los tres nodos. En la configuración actual de E3, con  $f = 1$  (instancia única, sin réplicas),  $t_{\text{máx}} = 0$ : cualquier caída del nodo de base de datos deja al sistema completo sin disponibilidad, lo cual es precisamente la limitación que motiva el trabajo planificado para E4 (Sección 9).

## 4.2. Teorema CAP y modelo PACELC

El teorema CAP, formulado por Brewer y demostrado formalmente por Gilbert y Lynch [11], establece que ante una partición de red ( $P$ ) un sistema distribuido debe elegir entre consistencia ( $C$ ) y disponibilidad ( $A$ ); no puede garantizar ambas simultáneamente. El teorema CAP describe únicamente el comportamiento *durante* una partición; el modelo PACELC de Abadi [12] lo extiende con un segundo eje que aplica incluso en operación normal: en ausencia de partición ( $E$ , *else*), el sistema debe elegir entre latencia ( $L$ ) y consistencia ( $C$ ). Un clúster CockroachDB se clasificaría como **PC/EC**: prioriza consistencia serializable tanto ante particiones como en ausencia de ellas, aceptando la latencia adicional que impone la confirmación por mayoría Raft en cada escritura [8]. La instancia única de PostgreSQL efectivamente desplegada en E3, en cambio, se clasifica como **CA**: al no existir partición posible (un solo nodo), el sistema es consistente y disponible mientras el nodo esté en pie, pero no ofrece ninguna garantía de tolerancia a particiones ni a la caída del propio nodo — exactamente el comportamiento de punto único de falla señalado desde E2. Esta elección de diseño hacia PC/EC seguirá siendo la apropiada para el dominio de calificaciones del PFC una vez migrado el clúster, donde una escritura perdida o duplicada (por ejemplo, una nota registrada dos veces por inconsistencia eventual) tiene un costo académico mayor que una latencia adicional de unos pocos milisegundos por transacción.

### 4.3. Consenso distribuido: Raft

Raft descompone el problema del consenso distribuido en tres subproblemas independientes [9]: *elección de líder* (los nodos alternan entre los estados *seguidor*, *candidato* y *líder*, y un candidato gana la elección de un término al obtener votos de una mayoría estricta de nodos), *replicación de registros* (el líder es el único que acepta escrituras del cliente y las propaga como entradas de *log* a los seguidores) y *seguridad* (una entrada de *log* solo se considera *comprometida* — y por tanto visible para el cliente — cuando ha sido replicada en una mayoría estricta de nodos del grupo, garantía que impide que un líder posterior sobrescriba datos ya confirmados). En CockroachDB, cada *rango* de datos — la unidad de fragmentación física, no necesariamente coincidente con el fragmento lógico de la Ecuación (1) — constituiría un grupo Raft independiente con su propio líder, lo que permitiría que distintos rangos (por ejemplo, calificaciones de distintos trimestres) eligieran líderes en nodos distintos y balancearan la carga de escritura entre los tres nodos del clúster. Como se documenta en la Sección 9, esta capa de consenso no fue desplegada en E3 y no existe, por tanto, evidencia experimental propia sobre elección de líder o tolerancia a fallos de nodo de base de datos; la descripción anterior es teórica y se retoma como objetivo de implementación en E4.

### 4.4. Modelo de programación paralela y leyes de escalabilidad

El modelo RDD de Spark abstrae una colección distribuida e inmutable con tres propiedades centrales [13]: *linaje* (cada RDD registra la secuencia de transformaciones que lo produjo, en lugar de materializar resultados intermedios), *evaluación perezosa* (las transformaciones — `filter`, `groupBy`, `join` — no se ejecutan hasta que una acción como `write` o `count` las dispare) y *tolerancia a fallos por recómputo* (una partición perdida se reconstruye a partir de su linaje en lugar de leerse desde una réplica física). Sobre este modelo se evaluaría el *speedup*  $S(N)$  obtenido al paralelizar el pipeline analítico entre  $N$  núcleos frente a su ejecución secuencial ( $N = 1$ ), y la *eficiencia* asociada:

$$E(N) = \frac{S(N)}{N}. \quad (4)$$

La Ley de Amdahl [16] predice el límite superior de  $S(N)$  para un problema de tamaño *fijo*, en función de la fracción paralelizable  $p$  medida sobre la ejecución secuencial:

$$S(N) = \frac{1}{(1 - p) + \frac{p}{N}}, \quad (5)$$

de donde se sigue que, incluso con  $N \rightarrow \infty$ , el speedup está acotado por  $S_{\text{máx}} = 1/(1 - p)$ : si solo el 80 % del pipeline es paralelizable ( $p = 0,8$ ), el speedup máximo teórico es  $5\times$ , sin importar cuántos núcleos se añadan. Gustafson [17] cuestiona la premisa de tamaño fijo y propone en su lugar medir la fracción serial  $s$  sobre la ejecución paralela y dejar que el volumen de datos escale con  $N$ , obteniendo el speedup escalado:

$$S(N) = N - s(N - 1) = s + N(1 - s). \quad (6)$$

La diferencia no es solo algebraica: mientras que la Ecuación (5) predice rendimientos decrecientes al añadir núcleos sobre un dataset fijo, la Ecuación (6) es lineal en  $N$  cuando el volumen de datos procesado crece proporcionalmente a los recursos disponibles — el escenario más realista para un pipeline analítico que, con el tiempo, procesará más calificaciones a medida que la unidad educativa crezca. El protocolo experimental descrito en el Módulo F de la guía mide  $p$  y  $s$  de forma independiente a partir de los tiempos por transformación  $t_i$  registrados en cada corrida; como se explica en la Sección 7, este protocolo no pudo ejecutarse en E3 porque el pipeline PySpark que lo alimentaría aún no existe, y queda como objetivo central de la Entrega 4.

## 5. Diseño del esquema distribuido

### 5.1. Estrategia de fragmentación efectivamente adoptada

A diferencia de lo exigido por la guía (fragmentación horizontal por `id_periodo` sobre un clúster CockroachDB, Sección 4.1), la estrategia realmente implementada en E3 es *fragmentación vertical por microservicio* sobre una única instancia de PostgreSQL 17 alojada en AWS EC2, tal como se muestra en la Figura 1. La instancia aloja cuatro esquemas lógicos independientes:

- **sga\_principal**: autenticación, usuarios, roles y token JWT de *single sign-on* (propietario: **sga-principal**).
- **sga\_secretaria**: matrículas y representantes legales (escritor único autorizado: **sga-principal**, por la dependencia de integridad referencial heredada de E2).
- **sga\_docente**: asistencias y calificaciones (propietario: **micro-docente**, expuesto a **sga-principal** únicamente vía el contrato gRPC descrito en la Sección 6).

- **sga\_soporte**: tickets y bitácora de incidencias (propietario: soporte-técnico).

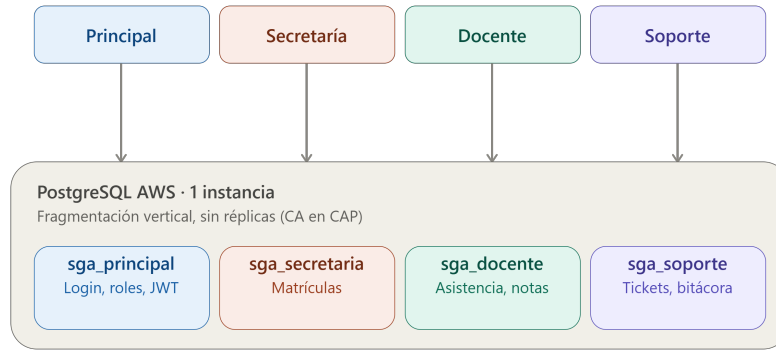


Figura 1: Esquema distribuido efectivamente implementado en E3: cuatro microservicios contra una única instancia PostgreSQL en AWS, fragmentada verticalmente en cuatro esquemas independientes por servicio.

Esta estrategia cumple parcialmente las reglas de corrección de Özsu y Valduriez [4]: la *disjunción* se satisface de forma trivial (cada tabla pertenece a exactamente un esquema, sin solapamiento), y la *completitud* y *reconstrucción* se satisfacen a nivel de esquema completo. Sin embargo, esta fragmentación **no** es la fragmentación horizontal exigida por el Módulo B de la guía: ninguna tabla individual (por ejemplo, **calificaciones**) fue particionada en fragmentos  $R_1, \dots, R_n$  por un predicado como **id\_periodo**; toda la tabla reside íntegra en el esquema **sga\_docente** de un único nodo físico. En consecuencia, tampoco se aplicó colocalización explícita ni factor de replicación  $f > 1$ : el sistema opera con  $f = 1$  (Sección 4.2).

## 5.2. Registro de decisión arquitectónica (ADR-003)

Siguiendo la plantilla de Nygard exigida por el Módulo B, se documenta la decisión efectivamente tomada:

### ADR-003 — Fragmentación vertical por microservicio (E3).

**Estado:** aceptada para E3; superada por ADR-005 (fragmentación horizontal con CockroachDB) planificada para E4.

**Contexto:** la guía de E3 exige un clúster distribuido con fragmentación horizontal y consenso; el equipo, por restricciones de tiempo, priorizó consolidar el contrato gRPC y la autenticación

JWT de *single sign-on* entre los cuatro portales antes de abordar la migración de persistencia.

**Decisión:** mantener una única instancia PostgreSQL en AWS EC2, fragmentada verticalmente en cuatro esquemas por servicio, con `sga-principal` como único escritor de `sga_secretaria`.

**Consecuencias:** (+) aislamiento lógico real de los datos de cada servicio sin multiplicar infraestructura; (+) migración de esquema simplificada al no requerir reparto de datos entre nodos; (−) persiste el punto único de falla identificado desde E2 (Sección 4.2); (−) ninguna tabla individual está fragmentada horizontalmente, por lo que las consultas de agregación por período siguen ejecutándose sobre el conjunto completo de la tabla en un solo nodo.

### 5.3. Seguridad de credenciales

De forma complementaria a la fragmentación, el equipo endureció el manejo de credenciales en los cuatro esquemas: las contraseñas se almacenan con BCrypt (costo 10) [21] en lugar de texto plano en cualquiera de los esquemas, y se exige cambio de contraseña obligatorio en el primer ingreso de cada usuario — una medida de higiene de seguridad independiente de la estrategia de fragmentación, pero relevante para la integridad del dominio distribuido en su conjunto.

## 6. Verificación experimental de la arquitectura implementada

Esta sección sustituye al Módulo C/D de la guía (despliegue y verificación de tolerancia a fallos de un clúster CockroachDB de tres nodos), que **no se ejecutó** porque el clúster correspondiente no fue desplegado (Sección 9). En su lugar, se documenta la verificación que sí se realizó sobre la pieza que efectivamente se consolidó en E3: el contrato gRPC compartido y la autenticación JWT de *single sign-on*.

### 6.1. Contrato gRPC compartido

El contrato `DocenteService` se define en un único archivo `docente.proto`, compilado tanto para el cliente Java (`sga-principal`) como para el servidor Python (`micro-docente`, vía `grpcio`), garantizando que ambos lados del contrato queden tipados desde la misma fuente:

Listing 1: Extracto de `docente.proto`: contrato compartido entre `sga-principal` (cliente) y `micro-docente` (servidor).

```
1 service DocenteService {
2     rpc ObtenerCalificaciones (ObtenerCalificacionesRequest
3         )
4         returns (ObtenerCalificacionesResponse);
5     rpc CalcularPromedioFormativo (PromedioRequest)
6         returns (PromedioResponse);
7     rpc CalcularPromedioFinal (PromedioRequest)
8         returns (PromedioResponse);
9     rpc RegistrarCalificacion (RegistrarCalificacionRequest
10        )
11        returns (RegistrarCalificacionResponse);
12     rpc ConvertirACualitativa (ConvertirACualitativaRequest
13        )
14        returns (ConvertirACualitativaResponse);
15 }
16
17 message Calificacion {
18     int64 id_calificacion = 1;
19     int64 id_actividad = 2;
20     int64 id_matricula = 3;
21     double nota = 4;
22     string nota_cualitativa = 5;
23     int32 registrado_por = 6;
24 }
```

El canal `gRPC` opera en los puertos 9091/9092, separado del canal `REST` de `micro-docente` (puerto 8081), lo que permite verificar su independencia (Sección 6.3).

## 6.2. Autenticación JWT de single sign-on

El token JWT, firmado con HMAC-SHA256 por `sga-principal` siguiendo RFC 7519 [1], se valida en los cuatro portales (Principal, Secretaría, Docente y Soporte) sin que el usuario deba autenticarse de nuevo al cambiar de portal. Se verificó manualmente que un mismo token, obtenido en el login del portal Docente, es aceptado por los cuatro backends dentro de su ventana de expiración.

### 6.3. Prueba de independencia del canal gRPC respecto de la API REST

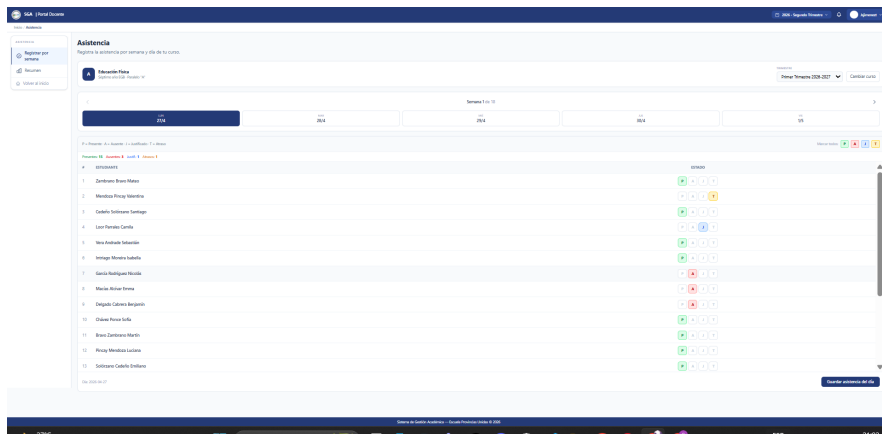
Para verificar la afirmación del Resumen (Sección 1) de que el canal gRPC opera de forma independiente de la disponibilidad de la API REST de **micro-docente**, se siguió el siguiente procedimiento:

1. Con el sistema completo desplegado vía **docker compose**, se inicia sesión como docente y se navega al módulo de asistencia (portal React en **localhost:3000**), confirmando que las peticiones de negocio (registro de asistencia) viajan por gRPC y no por la API REST de Django.
2. Se detiene únicamente el proceso REST de **micro-docente** (Django Rest Framework), dejando el servidor gRPC (**grpcio**) activo en el mismo contenedor.
3. Se repite el registro de asistencia grupal desde **sga-principal**: la operación se completa con éxito, confirmando que el flujo

React → sga-principal → gRPC → micro-docente → BD

no depende de la disponibilidad de la capa REST.

4. Se consulta directamente el esquema **sga\_docente** y se confirma la persistencia de las 20 filas de asistencia correspondientes al curso de prueba (Figura 2).



| STUDENT                      | Presente | Ausente | Justificado | Atraso |
|------------------------------|----------|---------|-------------|--------|
| 1. Zenteno Bruna Maria       | ✓        |         |             |        |
| 2. Mendez Pina Valeria       | ✓        |         |             |        |
| 3. Gálvez Solano Santiago    | ✓        |         |             |        |
| 4. Lora Pineda Camila        | ✓        |         |             |        |
| 5. Neri Gabriela Victoria    | ✓        |         |             |        |
| 6. Arango Medina Luciana     | ✓        |         |             |        |
| 7. García Rodríguez Mariana  | ✓        |         |             |        |
| 8. Méndez Ariana Elena       | ✓        |         |             |        |
| 9. Delgado Calvoa Brigidita  | ✓        |         |             |        |
| 10. Olvera Pineda Julia      | ✓        |         |             |        |
| 11. Bravo Castellano Marlen  | ✓        |         |             |        |
| 12. Arango Medina Luciana    | ✓        |         |             |        |
| 13. Solano Gabriela Estrella | ✓        |         |             |        |

Figura 2: Portal Docente: registro de asistencia por semana para el curso de Educación Física (Séptimo año EGB, Paralelo “A”), con el conteo de presentes, ausentes, justificados y atrasos calculado en tiempo real.

Esta prueba corresponde, en alcance, a una verificación de *tolerancia a fallos de un componente de aplicación* (la API REST), y no reemplaza la verificación de tolerancia a fallos de *nodos de base de datos* exigida por el Módulo D sobre el clúster CockroachDB, que no fue posible ejecutar al no existir dicho clúster.

## 7. Pipeline analítico paralelo

El Módulo E de la guía exige un pipeline PySpark con al menos cinco transformaciones (filtrado, *join* entre tablas colocalizadas, agregación con ventanas, transformación de tipos temporales y una operación de `ml`) sobre datos extraídos del clúster distribuido, junto con un baseline secuencial en `pandas` para el contraste de la Ley de Amdahl. **Este componente no fue implementado en E3.**

La razón es directa: el pipeline analítico está diseñado para leer, vía conector JDBC particionado, un clúster con fragmentación horizontal real (Sección 4.1); al no existir ese clúster (Sección 5), el equipo decidió no construir un pipeline Spark sobre la instancia única de PostgreSQL, ya que hacerlo habría medido el comportamiento de Spark contra un origen de datos no representativo del escenario que la guía busca evaluar (rendimiento del procesamiento paralelo frente a fragmentación real de los datos), y hubiera requerido rehacer la extracción una vez migrado el clúster en E4.

Se planifica para la Entrega 4 — una vez desplegado el clúster CockroachDB de la Sección 5 — implementar `spark/pipeline.py` y `spark/baseline.py` siguiendo el esqueleto sugerido por la guía, aplicado sobre la tabla `calificaciones` con las cinco transformaciones exigidas y el protocolo experimental completo de la Sección 8.

## 8. Protocolo y resultados

### 8.1. Alcance de la medición realizada

A diferencia del protocolo estadístico completo del Módulo F (variable independiente  $N \in \{1, 2, 4, 8\}$  núcleos de Spark,  $r = 10$  repeticiones por nivel, intervalo de confianza al 95 %), que depende del pipeline PySpark descrito en la Sección 7 y por tanto no pudo ejecutarse, el equipo sí realizó una medición comparativa — de alcance más acotado — entre las rutas gRPC y REST para las dos operaciones más frecuentes del dominio docente. Esta medición se reporta a continuación con la caveat explícita de que **no** sigue el diseño de bloque completo con  $r = 10$  repeticiones ni reporta desviación típica o intervalo de



confianza; es una medición preliminar de tiempo de respuesta observado, no un experimento controlado en el sentido de Wohlin et al. [19].

## 8.2. Resultados observados

Cuadro 1: Latencia observada por operación y ruta de comunicación.

| Operación                 | Ruta | Latencia observada (ms) |
|---------------------------|------|-------------------------|
| RegistrarAsistenciaGrupal | gRPC | 51                      |
| GetMisAsignaciones        | gRPC | 42                      |
| /mis-asignaciones         | REST | 145                     |

Para la operación comparable en ambas rutas, la razón de latencia observada es de  $145/51 \approx 2,8\times$ : el canal **gRPC** respondió aproximadamente tres veces más rápido que la ruta **REST** equivalente en las condiciones de prueba (contenedor local, curso de 20 estudiantes; el sistema en producción gestiona 677 estudiantes, 13 docentes, 12 grados y 3 paralelos). Esta diferencia es consistente con la literatura sobre serialización binaria **Protobuf** frente a **JSON** sobre **HTTP/1.1** [2], aunque — se insiste — no se controlaron aquí variables como el calentamiento de la JVM, el ruido del sistema anfitrión ni la caché del sistema operativo, que sí exige el protocolo del Módulo F para resultados defendibles como evidencia estadística.

## 8.3. Protocolo formal pendiente

El protocolo completo — hipótesis  $H_1$  vs.  $H_0$ ,  $r = 10$  repeticiones descartando calentamiento y enfriamiento, prueba  $t$  pareada o Wilcoxon según normalidad, intervalo de confianza al 95 % — queda pendiente y se aplicará en E4 tanto al pipeline **Spark** (Sección 7) como, de forma extendida, a la comparación **gRPC** vs. **REST** reportada aquí de forma preliminar. Para esta última, el equipo planea adoptar además una metodología de *benchmarking* de servicios más cercana a la de Cooper et al. [22], generando carga sintética creciente sobre ambas rutas en lugar de una única medición puntual por operación.

## 9. Discusión

Esta sección discute los resultados reportados siguiendo las recomendaciones de reporte de estudios de caso en ingeniería de software de Runeson y

Höst [23]: se separan explícitamente las limitaciones de alcance (qué no se implementó), las amenazas a la validez de lo que sí se midió, y la relación de continuidad con la entrega anterior, en lugar de presentar los hallazgos como un experimento cerrado.

## 9.1. Limitaciones de esta entrega

Esta entrega presenta cuatro limitaciones explícitas frente a lo exigido por la guía de la asignatura:

1. **Ausencia de fragmentación horizontal.** La persistencia permanece fragmentada únicamente de forma vertical (Sección 5); ninguna tabla fue particionada por `id_periodo` ni colocalizada por `id_estudiante`.
2. **Ausencia de replicación y consenso.** No se desplegó el clúster CockroachDB de tres nodos exigido por el Módulo C; el sistema opera con factor de replicación  $f = 1$  y, por tanto, sin tolerancia a la caída del nodo de base de datos (Sección 4.2).
3. **Ausencia del pipeline analítico paralelo.** No se implementó el pipeline PySpark del Módulo E (Sección 7), y en consecuencia tampoco se dispone de mediciones propias de speedup para contrastar las leyes de Amdahl y Gustafson (Sección 4.4).
4. **Protocolo experimental incompleto.** La única medición cuantitativa reportada (Sección 8) no sigue el diseño de bloque completo ni reporta intervalos de confianza, por lo que debe leerse como indicativa y no como evidencia estadísticamente rigurosa.

## 9.2. Amenazas a la validez

**Internas.** (i) La medición de latencia de la Tabla 1 se realizó sin repeticiones controladas ni descarte de calentamiento, por lo que puede estar sesgada por el estado transitorio de la JVM o del intérprete Python en el momento de la prueba. (ii) La prueba de independencia gRPC/REST (Sección 6.3) se ejecutó una sola vez de forma manual, sin automatización que garantice reproducibilidad exacta. (iii) El entorno de prueba corrió en un único host de desarrollo, sin aislamiento de recursos frente a otros procesos del sistema anfitrión.

**Externas.** (i) Los resultados de latencia se midieron sobre un curso de 20 estudiantes; no está garantizado que la razón gRPC/REST observada se mantenga con cargas mayores (por ejemplo, matrícula completa de 677

estudiantes). (ii) El comportamiento observado corresponde a un entorno de contenedores locales sobre una única máquina; no se ha probado bajo latencia de red real entre nodos distintos, condición que sí aplicará una vez desplegado el clúster CockroachDB distribuido geográfica o físicamente en E4.

### 9.3. Relación con la Entrega 2

La decisión de priorizar la capa de comunicación (gRPC+JWT) sobre la capa de persistencia distribuida es coherente con el estado heredado de E2: en E2 la comunicación entre procesos ya estaba prototipada pero no verificada bajo fallos parciales, mientras que la persistencia mono-nodo, aunque señalada como limitación, seguía siendo funcional para la demostración de negocio. El equipo optó por cerrar primero la pieza que condicionaba la usabilidad demostrable del sistema (SSO real entre los cuatro portales) antes de abordar una migración de base de datos que, de quedar a medio hacer, habría dejado el sistema completo inoperable para la defensa oral.

## 10. Conclusiones e integración con Entrega 4

Este trabajo consolidó y verificó experimentalmente la capa de comunicación distribuida entre los cuatro microservicios del SGA: un contrato gRPC compartido entre `sga-principal` y `micro-docente`, autenticación JWT de *single sign-on* válida en los cuatro portales, e independencia verificada del canal gRPC respecto de la disponibilidad de la API REST de docente. La medición preliminar de latencia (Tabla 1) sugiere una ventaja de aproximadamente  $2,8\times$  de gRPC sobre REST para la operación más frecuente del dominio, aunque esta cifra debe confirmarse con el protocolo estadístico formal del Módulo F.

En contraste, este trabajo **no** alcanzó los dos componentes que la guía identifica como núcleo de la Entrega 3: un clúster distribuido con fragmentación horizontal, replicación y consenso (CockroachDB/Raft), y un pipeline analítico paralelo en Apache Spark contrastado contra las leyes de Amdahl y Gustafson. Ambos se documentan aquí como limitaciones explícitas (Sección 9) y constituyen la prioridad de la Entrega 4.

Los artefactos de esta entrega que se reutilizarán **sin modificación** en E4 son: (i) el contrato `docente.proto` y el canal gRPC 9091/9092; (ii) el esquema de autenticación JWT HMAC-SHA256 de *single sign-on*; (iii) los cuatro esquemas lógicos por servicio (`sga_principal`, `sga_secretaria`, `sga_docente`, `sga_soporte`), que se migrarán como namespaces dentro del clúster CockroachDB sin cambiar su modelo de datos; y (iv) el `docker-compose.yml` base, al que se añadirán los servicios `crdb-1..3` y `crdb-init` descritos en el

Módulo C de la guía. Sobre esta base, la Entrega 4 debe: desplegar el clúster de tres nodos y aplicar la fragmentación horizontal por `id_periodo`; ejecutar la prueba de tolerancia a fallos con caída controlada de uno y de dos nodos; implementar el pipeline `PySpark` con sus cinco transformaciones y el baseline `pandas`; y correr el protocolo experimental completo ( $r = 10$ , IC 95 %) tanto sobre el pipeline analítico como, de forma extendida, sobre la comparación gRPC/REST iniciada en esta entrega.

## Referencias

- [1] M. B. Jones, J. Bradley, and N. Sakimura, “JSON Web Token (JWT),” Tech. Rep. RFC 7519, IETF, 2015.
- [2] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, Irvine, CA, USA, 2000.
- [3] S. Newman, *Building Microservices: Designing Fine-Grained Systems*. Sebastopol, CA, USA: O’Reilly Media, 2 ed., 2021.
- [4] M. T. Özsu and P. Valduriez, *Principles of Distributed Database Systems*. Cham, Switzerland: Springer, 4 ed., 2020.
- [5] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA, USA: O’Reilly Media, 2017.
- [6] J. C. Corbett *et al.*, “Spanner: Google’s globally-distributed database,” in *Proc. 10th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pp. 251–264, 2012.
- [7] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Vosshall, and W. Vogels, “Dynamo: Amazon’s highly available key-value store,” in *Proc. 21st ACM SIGOPS Symposium on Operating Systems Principles (SOSP)*, pp. 205–220, 2007.
- [8] R. Taft, I. Sharif, A. Matei, N. VanBenschoten, J. Lewis, T. Grieger, K. Niemi, A. Woods, A. Birzin, R. Poss, P. Bardea, A. Ranade, B. Darnell, B. Gruneir, J. Jaffray, L. Zhang, and P. Mattis, “CockroachDB: The resilient geo-distributed SQL database,” in *Proc. 2020 ACM SIGMOD International Conference on Management of Data*, pp. 1493–1509, 2020.

- [9] D. Ongaro and J. Ousterhout, “In search of an understandable consensus algorithm,” in *Proc. USENIX Annual Technical Conference (USENIX ATC)*, pp. 305–319, 2014.
- [10] E. Brewer, “CAP twelve years later: How the “rules” have changed,” *IEEE Computer*, vol. 45, no. 2, pp. 23–29, 2012.
- [11] S. Gilbert and N. Lynch, “Brewer’s conjecture and the feasibility of consistent, available, partition-tolerant web services,” *ACM SIGACT News*, vol. 33, no. 2, pp. 51–59, 2002.
- [12] D. J. Abadi, “Consistency tradeoffs in modern distributed database system design: CAP is only part of the story,” *IEEE Computer*, vol. 45, no. 2, pp. 37–42, 2012.
- [13] M. Zaharia, M. Chowdhury, T. Das, A. Dave, J. Ma, M. McCauley, M. J. Franklin, S. Shenker, and I. Stoica, “Resilient distributed datasets: A fault-tolerant abstraction for in-memory cluster computing,” in *Proc. 9th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, pp. 15–28, 2012.
- [14] B. Chambers and M. Zaharia, *Spark: The Definitive Guide: Big Data Processing Made Simple*. Sebastopol, CA, USA: O’Reilly Media, 2018.
- [15] S. Salloum, R. Dautov, X. Chen, P. X. Peng, and J. Z. Huang, “Big data analytics on apache spark,” *International Journal of Data Science and Analytics*, vol. 1, no. 3–4, pp. 145–164, 2016.
- [16] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” in *Proc. AFIPS Spring Joint Computer Conference*, pp. 483–485, 1967.
- [17] J. L. Gustafson, “Reevaluating amdahl’s law,” *Communications of the ACM*, vol. 31, no. 5, pp. 532–533, 1988.
- [18] R. Jain, *The Art of Computer Systems Performance Analysis: Techniques for Experimental Design, Measurement, Simulation, and Modeling*. Wiley, 1991.
- [19] C. Wohlin, P. Runeson, M. Höst, M. C. Ohlsson, B. Regnell, and A. Wesslén, *Experimentation in Software Engineering*. Berlin, Germany: Springer, 2012.
- [20] Cockroach Labs, “CockroachDB documentation v23.2,” 2024. Online documentation.

- [21] N. Provos and D. Mazières, “A future-adaptable password scheme,” in *Proc. USENIX Annual Technical Conference (USENIX ATC)*, pp. 81–92, 1999.
- [22] B. F. Cooper, A. Silberstein, E. Tam, R. Ramakrishnan, and R. Sears, “Benchmarking cloud serving systems with YCSB,” in *Proc. 1st ACM Symposium on Cloud Computing (SoCC)*, pp. 143–154, 2010.
- [23] P. Runeson and M. Höst, “Guidelines for conducting and reporting case study research in software engineering,” *Empirical Software Engineering*, vol. 14, no. 2, pp. 131–164, 2009.